Formation Vue.js Moderne

Jour 2 — Réactivité & Architecture

Modules 5 à 9

Composition API · Vue Router · Pinia · Tests

📋 Sommaire — Jour 2

#	Module	Contenu	Slides
5	**Répartir en composants**	Architecture, patterns, réutilisation	4–18
6	**Composition API**	ref, reactive, computed, watch, provide	19–42
7	**Vue Router**	Routes, guards, lazy loading, params	43–64
8	**Pinia**	Stores, getters, actions, DevTools	65–85
9	**Tests & Qualité**	Vitest, Vue Test Utils, Cypress, CI/CD	86–100

> 🕐 Durée estimée : 7h de formation

Module 5

Répartir son code dans des composants

Architecture · Patterns · Réutilisation

Module 5 — Plan

1. Principes d'architecture composants
2. Patterns de composition
3. Composables — logique réutilisable
4. Validation et tests de composants
5. Bonnes pratiques de structuration
6. ✏️ Exercice

5.1 — Principes d'architecture

Règle d'or : Séparation des responsabilités

```
App.vue (racine)
├── layouts/
│   ├── DefaultLayout.vue      # mise en page générale
│   └── AuthLayout.vue         # mise en page auth
├── views/
│   ├── HomeView.vue            # page complète
│   └── UserView.vue
└── components/
    ├── ui/                     # composants génériques
    │   ├── AppButton.vue
    │   └── AppInput.vue
    ├── features/               # composants métier
    │   └── user/
    │       ├── UserCard.vue
    │       └── UserList.vue
    └── icons/
```

5.2 — Quand découper un composant ?

Signal pour découper :

- Le template dépasse **50 lignes**
- Une logique est dupliquée dans 2+ endroits
- Une partie peut être **réutilisée** ailleurs
- Un composant gère plusieurs **responsabilités**

```
<!-- ❌ Trop monolithique -->
<template>
  <!-- 200 lignes de template -->
  <!-- Navigation + Formulaire + Liste + Modal -->
</template>

<!-- ✅ Décomposé -->
<template>
  <AppNavigation />
```

5.3 — Composables : logique réutilisable

```
// composables/useCounter.ts
import { ref, computed } from 'vue'

export function useCounter(initialValue = 0) {
  const count = ref(initialValue)

  const double   = computed(() => count.value * 2)
  const isZero   = computed(() => count.value === 0)

  function increment() { count.value++ }
  function decrement() { count.value-- }
  function reset()     { count.value = initialValue }

  return { count, double, isZero, increment, decrement, reset }
}

// Utilisation dans n'importe quel composant
const { count, increment, reset } = useCounter(10)
```

5.4 — Composable : useFetch

```ts
// composables/useFetch.ts
import { ref, onMounted } from 'vue'

export function useFetch<T>(url: string) {
  const data    = ref<T | null>(null)
  const loading = ref(true)
  const error   = ref<string | null>(null)

  async function fetchData() {
    loading.value = true
    error.value   = null
    try {
      const res  = await fetch(url)
      if (!res.ok) throw new Error(`HTTP ${res.status}`)
      data.value = await res.json()
    } catch (e) {
      error.value = (e as Error).message
    } finally {
      loading.value = false
    }
  }

  onMounted(fetchData)
  return { data, loading, error, refetch: fetchData }
}
```

5.5 — Composable : useLocalStorage

```ts
// composables/useLocalStorage.ts
import { ref, watch } from 'vue'

export function useLocalStorage<T>(key: string, defaultValue: T) {
  const stored = localStorage.getItem(key)

  const value = ref<T>(
    stored !== null ? JSON.parse(stored) : defaultValue
  )

  // Synchronise la ref avec localStorage à chaque changement
  watch(
    value,
    (newVal) => localStorage.setItem(key, JSON.stringify(newVal)),
    { deep: true }
  )

  return value
}

// Dans un composant :
const panier = useLocalStorage('panier', [])
// panier persiste entre les sessions 🎉
```

5.6 — Composable : useDebounce

```ts
// composables/useDebounce.ts
import { ref, watch } from 'vue'

export function useDebounce<T>(value: Ref<T>, delay = 300) {
  const debouncedValue = ref<T>(value.value) as Ref<T>

  watch(value, (newVal) => {
    const timer = setTimeout(() => {
      debouncedValue.value = newVal
    }, delay)
    return () => clearTimeout(timer)
  })

  return debouncedValue
}

// Utilisation — recherche différée
const recherche        = ref("")
const rechercheDebounced = useDebounce(recherche, 500)

watch(rechercheDebounced, (query) => {
  // Appel API déclenché seulement après 500ms d'inactivité
  fetchResults(query)
})
```

5.7 — Pattern : Container / Presentational

```
<!-- Container : gère la logique et les données -->
<!-- UserListContainer.vue -->
<script setup lang="ts">
const { data: users, loading } = useFetch<User[]>('/api/users')
function handleDelete(id: number) { /* ... */ }
</script>
<template>
  <UserList :users="users" :loading="loading"
            @delete="handleDelete" />
</template>

<!-- Presentational : affiche uniquement, sans logique -->
<!-- UserList.vue -->
<script setup lang="ts">
defineProps<{ users: User[]; loading: boolean }>()
defineEmits<{ delete: [id: number] }>()
</script>
<template>
  <div v-if="loading">Chargement...</div>
  <ul v-else>
    <li v-for="u in users" :key="u.id">{{ u.nom }}</li>
  </ul>
</template>
```

5.8 — Organisation des imports

```
// Ordre recommandé des imports dans <script setup>
// 1. Vue core
import { ref, computed, onMounted } from 'vue'

// 2. Vue Router / Pinia
import { useRouter, useRoute } from 'vue-router'
import { useAuthStore } from '@/stores/auth'

// 3. Composables internes
import { useFetch } from '@/composables/useFetch'

// 4. Composants
import UserCard from '@/components/UserCard.vue'
import AppButton from '@/components/ui/AppButton.vue'

// 5. Types
import type { User } from '@/types/user'

// 6. Utils
import { formatDate } from '@/utils/date'
```

5.9 — Récapitulatif Module 5

Pattern	Quand l'utiliser
Découpage	Template > 50 lignes, logique dupliquée
Composable	Logique réutilisable entre composants
Container/Presentational	Séparer données et affichage
Alias `@`	Imports toujours depuis `src/`
Barrel exports	`index.ts` par dossier de composants

✏️ Exercice — Module 5

Exercice 4.3 — Décomposition d'une interface *(suite)*

> Reprendre le projet ToDoList et découper en composants :
> `TodoList.vue`, `TodoItem.vue`, `TodoForm.vue`, `TodoFilters.vue`
> Extraire la logique dans un composable `useTodos.ts`

> 🕐 Durée estimée : 30 minutes

Module 6

Gestion de l'état avec la Composition API

ref · reactive · computed · watch · provide/inject

Module 6 — Plan

1. `ref` en profondeur
2. `reactive` et ses limites
3. `computed` properties
4. `watch` et `watchEffect`
5. `provide` / `inject`
6. Patterns de state management locaux
7. ✏️ Exercice

6.1 — `ref` : valeurs réactives primitives

```
import { ref, isRef, unref } from 'vue'

// Créer une ref
const count   = ref(0)
const message = ref("Hello")
const visible = ref(false)

// Lire / modifier (nécessite .value dans <script>)
console.log(count.value)   // 0
count.value++
count.value = 42

// Dans le template : PAS besoin de .value
// <p>{{ count }}</p>  ← automatiquement déballé

// Utilitaires
isRef(count)     // true
unref(count)     // 0 — retourne .value si ref, sinon la valeur

// Typage explicite
const user = ref<User | null>(null)
```

6.2 — `reactive` : objets réactifs

```
import { reactive, toRefs } from 'vue'

const etat = reactive({
  utilisateur: null as User | null,
  chargement:  false,
  erreur:      null as string | null,
  pagination:  { page: 1, total: 0, parPage: 10 }
})

// Pas besoin de .value pour reactive !
etat.chargement = true
etat.pagination.page++

// ⚠️ Limites de reactive :
// Ne PAS réassigner l'objet entier
etat = { ...etat, chargement: false }  // ❌ perd la réactivité

// Utiliser toRefs pour déstructurer sans perdre la réactivité
const { chargement, erreur } = toRefs(etat)  // ✅ reste réactif
```

6.3 — `ref` vs `reactive` : quel choisir ?

	`ref`	`reactive`
Primitifs	✅ idéal	❌
Objets	✅ fonctionne	✅ idéal
Tableaux	✅	✅
Déstructuration	✅ naturelle	⚠️ perd réactivité
Réassignation	✅ `.value = nouveau`	❌ perdre réactivité
Template	Auto-déballé	Direct

> 💡 **Recommandation officielle Vue 3 :** préférer `ref` pour tout.
> C'est plus cohérent et évite les pièges de `reactive`.

6.4 — `computed` : valeurs dérivées

```ts
import { ref, computed } from 'vue'

const prix      = ref(100)
const quantite  = ref(3)
const remise    = ref(10) // %

// computed simple (lecture seule)
const sousTotal = computed(() => prix.value * quantite.value)
const total     = computed(
  () => sousTotal.value * (1 - remise.value / 100)
)

// computed en lecture/écriture
const celsius = ref(20)
const fahrenheit = computed({
  get: () => celsius.value * 9/5 + 32,
  set: (val: number) => { celsius.value = (val - 32) * 5/9 }
})

// Accès
console.log(total.value)   // 270
fahrenheit.value = 68      // celsius.value devient 20
```

6.5 — `watch` : surveiller les changements

```
import { ref, watch } from 'vue'

const query = ref("")
const page  = ref(1)

// Surveiller une ref
watch(query, (newVal, oldVal) => {
  console.log(`${oldVal} → ${newVal}`)
  page.value = 1  // Remettre à la page 1 à chaque recherche
})

// Options du watcher
watch(query, fetchResults, {
  immediate: true,   // déclencher aussi au montage
  deep:      true,   // surveiller les changements profonds
  once:      true    // se déclenche une seule fois
})

// Surveiller plusieurs sources
watch([query, page], ([q, p]) => {
  fetchResults(q, p)
})
```

6.6 — `watchEffect` : réactivité automatique

```
import { ref, watchEffect } from 'vue'

const userId = ref(1)
const user   = ref(null)

// watchEffect suit automatiquement toutes les deps utilisées
watchEffect(async () => {
  // userId.value est lu → automatiquement surveillé
  const res = await fetch(`/api/users/${userId.value}`)
  user.value = await res.json()
})
// Se rejoue automatiquement quand userId change

// Stopper manuellement un watcher
const stop = watchEffect(() => {
  console.log(userId.value)
})
stop()  // stopper

// watchEffect vs watch
// watchEffect : immédiat, deps automatiques
// watch       : déclenché sur changement, accès newVal/oldVal
```

6.7 — `provide` / `inject`

```
// Parent (n'importe quel ancêtre)
// App.vue ou un composant parent
import { provide, ref } from 'vue'

const theme = ref<'light' | 'dark'>('light')

provide('theme', theme)  // injecter la ref elle-même

function basculerTheme() {
  theme.value = theme.value === 'light' ? 'dark' : 'light'
}
provide('basculerTheme', basculerTheme)

// Composant enfant (à n'importe quelle profondeur)
import { inject, Ref } from 'vue'

const theme         = inject<Ref<'light' | 'dark'>>('theme')
const basculerTheme = inject<() => void>('basculerTheme')
// ✅ Réactif : le changement de theme se propage
```

6.8 — Provide/Inject avec clés symboliques

```ts
// types/injection-keys.ts — clés typées
import type { InjectionKey, Ref } from 'vue'

export const THEME_KEY: InjectionKey<Ref<'light'|'dark'>> =
  Symbol('theme')
export const AUTH_KEY: InjectionKey<AuthContext> =
  Symbol('auth')

// Provider
import { THEME_KEY } from '@/types/injection-keys'
provide(THEME_KEY, theme)

// Consumer — entièrement typé !
import { THEME_KEY } from '@/types/injection-keys'
const theme = inject(THEME_KEY)  // Ref<'light'|'dark'>

// Avec valeur par défaut
const theme = inject(THEME_KEY, ref('light'))
```

6.9 — Pattern : store local avec composable

```ts
// composables/useShoppingCart.ts
import { ref, computed } from 'vue'

interface CartItem { id: number; nom: string; prix: number; qty: number }

const items = ref<CartItem[]>([])  // état partagé hors du composable

export function useShoppingCart() {
  const total = computed(
    () => items.value.reduce((sum, i) => sum + i.prix * i.qty, 0)
  )
  const count = computed(
    () => items.value.reduce((sum, i) => sum + i.qty, 0)
  )

  function ajouter(item: Omit<CartItem, 'qty'>) {
    const existing = items.value.find(i => i.id === item.id)
    existing ? existing.qty++ : items.value.push({ ...item, qty: 1 })
  }

  return { items, total, count, ajouter }
}
```

6.10 — `toRef` et `toValue`

```ts
import { reactive, toRef, toValue } from 'vue'

const etat = reactive({ count: 0, nom: "Alice" })

// toRef — créer une ref depuis une propriété réactive
const count = toRef(etat, 'count')
count.value++  // modifie aussi etat.count ✅

// toValue — obtenir la valeur d'une ref ou d'une valeur brute
function doubler(val: Ref<number> | number) {
  return toValue(val) * 2  // fonctionne dans les deux cas
}
doubler(count)   // 2
doubler(42)      // 84

// Pratique pour les composables qui acceptent refs ou valeurs
function useMesure(longueur: Ref<number> | number) {
  return computed(() => toValue(longueur) * 2.54)
}
```

6.11 — Récapitulatif Module 6

API	Usage	Syntaxe
`ref`	Valeur réactive (tout type)	`ref(0)`, `.value`
`reactive`	Objet réactif	`reactive({})`, direct
`computed`	Valeur dérivée, cachée	`computed(() => ...)`
`watch`	Réagir aux changements	`watch(src, handler)`
`watchEffect`	Effets automatiques	`watchEffect(() => ...)`
`provide`	Injecter en descendant	`provide(key, value)`
`inject`	Récupérer une injection	`inject(key)`
`toRefs`	Déstructurer reactive	`const {a} = toRefs(obj)`

✏️ Exercice — Module 6

Exercice 6.1 — ref vs reactive

Exercice 6.2 — computed

Exercice 6.3 — watch

Exercice 6.4 — Provide / Inject

> 🕐 Durée estimée : 1h00

Module 7

Vue Router et Navigation

Configuration · Guards · Lazy loading · Nested routes

Module 7 — Plan

1. Configuration du router
2. Liens et navigation déclarative
3. Paramètres de route
4. Navigation guards
5. Route meta fields
6. Lazy loading
7. Routes imbriquées
8. ✏️ Exercice

7.1 — Configuration de base

```
// src/router/index.ts
import { createRouter, createWebHistory } from 'vue-router'
import HomeView    from '@/views/HomeView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView
    },
    {
      path: '/about',
      name: 'about',
      component: () => import('@/views/AboutView.vue')  // lazy
    },
    {
      path: '/:pathMatch(.*)*',  // 404
      name: 'not-found',
      component: () => import('@/views/NotFoundView.vue')
    }
  ]
})

export default router
```

7.2 — `<RouterLink>` et navigation déclarative

```
<template>
  <!-- Lien simple -->
  <RouterLink to="/">Accueil</RouterLink>
  <RouterLink to="/about">À propos</RouterLink>

  <!-- Lien nommé -->
  <RouterLink :to="{ name: 'user', params: { id: 42 } }">
    Profil utilisateur
  </RouterLink>

  <!-- Avec query string et hash -->
  <RouterLink :to="{
    name: 'search',
    query: { q: 'vue', page: 1 },
    hash: '#resultats'
  }">
    Rechercher
  </RouterLink>

  <!-- Classe active automatique -->
  <!-- RouterLink ajoute .router-link-active et
       .router-link-exact-active automatiquement -->
</template>
```

7.3 — Navigation programmatique

```
import { useRouter, useRoute } from 'vue-router'

const router = useRouter()
const route  = useRoute()

// Naviguer
router.push('/')
router.push({ name: 'user', params: { id: 42 } })
router.push({ path: '/search', query: { q: 'vue' } })

// Remplacer (sans ajouter à l'historique)
router.replace({ name: 'home' })

// Navigation relative à l'historique
router.back()
router.forward()
router.go(-2)

// Lire les paramètres de la route actuelle
const userId = route.params.id          // string | string[]
const search = route.query.q            // string | string[]
const hash   = route.hash               // "#section"
const path   = route.path               // "/users/42"
```

7.4 — Routes dynamiques et paramètres

```
// Définition
const routes = [
  { path: '/users/:id',          name: 'user' },
  { path: '/users/:id/posts',    name: 'user-posts' },
  { path: '/files/:path(.*)',    name: 'file' },       // wildcard
  { path: '/items/:id(\\d+)',    name: 'item' },       // regex
]

// Dans le composant
const route  = useRoute()
const userId = route.params.id   // "42"

// Prop params — plus propre (option props: true)
// Dans la config :
{ path: '/users/:id', component: UserView, props: true }

// Dans le composant :
const props = defineProps<{ id: string }>()
// ✅ Plus besoin d'utiliser route.params
```

7.5 — Navigation Guards — global

```
// src/router/index.ts

// Guard global — s'exécute à chaque navigation
router.beforeEach(async (to, from) => {
  const authStore = useAuthStore()

  // Route nécessite authentification ?
  if (to.meta.requiresAuth && !authStore.isLoggedIn) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }

  // Route nécessite un rôle ?
  if (to.meta.role && authStore.user?.role !== to.meta.role) {
    return { name: 'forbidden' }
  }
})

// Après navigation
router.afterEach((to) => {
  // Exemple : mise à jour du titre de la page
  document.title = (to.meta.title as string) ?? 'Mon App'
})
```

7.6 — Navigation Guards — par composant

```
// Dans un composant
import { onBeforeRouteLeave, onBeforeRouteUpdate } from 'vue-router'

// Avant de quitter ce composant
onBeforeRouteLeave((to, from) => {
  if (formulaireModifie.value) {
    const ok = confirm("Des modifications non sauvegardées. Continuer ?")
    if (!ok) return false  // annule la navigation
  }
})

// Quand la route change mais le composant reste monté
// (ex: /users/1 → /users/2)
onBeforeRouteUpdate(async (to, from) => {
  if (to.params.id !== from.params.id) {
    // Recharger les données pour le nouvel ID
    await chargerUtilisateur(to.params.id as string)
  }
})
```

7.7 — Route meta fields

```ts
// Déclaration avec TypeScript
declare module 'vue-router' {
  interface RouteMeta {
    requiresAuth?: boolean
    role?: string
    title?: string
    transition?: string
  }
}

// Utilisation dans les routes
const routes = [
  {
    path: '/dashboard',
    meta: { requiresAuth: true, title: 'Tableau de bord' }
  },
  {
    path: '/admin',
    meta: { requiresAuth: true, role: 'admin', title: 'Admin' }
  },
  {
    path: '/login',
    meta: { title: 'Connexion' }
  }
]
```

7.8 — Lazy loading et code splitting

```
// ✅ Lazy loading — un chunk par route
const routes = [
  {
    path: '/',
    component: () => import('@/views/HomeView.vue')
  },
  {
    path: '/dashboard',
    // Nommer le chunk pour un meilleur debugging
    component: () => import(
      /* webpackChunkName: "dashboard" */
      '@/views/DashboardView.vue'
    )
  }
]

// Grouper plusieurs routes dans un seul chunk
const UserDetails  = () => import(/* webpackChunkName: "user" */ '@/views/UserDetails.vue')
const UserSettings = () => import(/* webpackChunkName: "user" */ '@/views/UserSettings.vue')
```

7.9 — Routes imbriquées (Nested Routes)

```
const routes = [
  {
    path: '/users/:id',
    component: UserLayout,    // composant avec <RouterView> intégré
    children: [
      { path: '',          component: UserProfile,  name: 'user' },
      { path: 'settings',  component: UserSettings, name: 'user-settings' },
      { path: 'posts',     component: UserPosts,    name: 'user-posts' },
    ]
  }
]

// UserLayout.vue doit contenir un <RouterView> pour ses enfants
```

```
<!-- UserLayout.vue -->
<template>
  <div class="user-layout">
```

7.10 — Transitions de route

```
<!-- App.vue — transition sur les changements de route -->
<template>
  <RouterView v-slot="{ Component, route }">
    <Transition
      :name="route.meta.transition || 'fade'"
      mode="out-in"
    >
      <component :is="Component" :key="route.path" />
    </Transition>
  </RouterView>
</template>

<style>
.fade-enter-active, .fade-leave-active { transition: opacity 0.3s; }
.fade-enter-from, .fade-leave-to       { opacity: 0; }

.slide-enter-active { transition: transform 0.3s; }
.slide-enter-from   { transform: translateX(100%); }
.slide-leave-to     { transform: translateX(-100%); }
</style>
```

7.11 — Récapitulatif Module 7

Configuration

- `createRouter` + `createWebHistory`
- Routes nommées
- Lazy loading `() => import()`

Navigation

- `<RouterLink :to="{}">`
- `router.push()` `.replace()`
- `route.params` `.query`

Guards

- `router.beforeEach()`
- `onBeforeRouteLeave()`
- `meta.requiresAuth`

✏️ Exercice — Module 7

Exercice 7.1 — Routes de base

Exercice 7.2 — Route dynamique

Exercice 7.3 — Guard d'authentification

Exercice 7.4 — Lazy loading

> 🕐 Durée estimée : 1h00

Module 8

Gestion d'état avec Pinia

Architecture · Stores · Actions · DevTools

Module 8 — Plan

1. Pourquoi Pinia ?
2. Créer un store
3. State, Getters, Actions
4. Stores composables (Setup syntax)
5. Composition entre stores
6. Persistance
7. Plugins et extensions
8. ✏️ Exercice

8.1 — Pourquoi Pinia ?

Problème sans Pinia :

```
A.vue → props → B.vue → props → C.vue → props → D.vue
```

Prop drilling — données passées à travers des composants intermédiaires qui n'en ont pas besoin.

Solution Pinia — state global partagé :

```
          Pinia Store
         /     |     \
    A.vue   C.vue   D.vue
```

> Pinia = successeur officiel de Vuex pour Vue 3.
> API plus simple, meilleur TypeScript, DevTools intégrés.

8.2 — Créer un store (Options API style)

```ts
// src/stores/counter.ts
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', {
  // state — données réactives du store
  state: () => ({
    count:   0,
    step:    1,
    history: [] as number[]
  }),

  // getters — propriétés calculées du store
  getters: {
    double:    (state) => state.count * 2,
    isPositive:(state) => state.count > 0,
  },

  // actions — méthodes (sync ou async)
  actions: {
    increment() {
      this.count += this.step
      this.history.push(this.count)
    }
  }
})
```

8.3 — Utiliser un store dans un composant

```
<script setup lang="ts">
import { storeToRefs } from 'pinia'
import { useCounterStore } from '@/stores/counter'

const store = useCounterStore()

// storeToRefs — destructurer le state/getters sans perdre la réactivité
const { count, double } = storeToRefs(store)

// Les actions peuvent être destructurées directement
const { increment, reset } = store
</script>

<template>
  <p>Count : {{ count }}</p>
  <p>Double : {{ double }}</p>
  <button @click="increment">+</button>
  <button @click="reset">Reset</button>
</template>
```

8.4 — Modifier le state

```
const store = useCounterStore()

// 1. Via une action (recommandé — traçable)
store.increment()

// 2. Modification directe (simple)
store.count = 10

// 3. $patch — modifier plusieurs propriétés en une fois
store.$patch({
  count: 100,
  step:  5
})

// 4. $patch avec callback — pour les opérations complexes
store.$patch((state) => {
  state.history.push(state.count)
  state.count = 0
})

// Réinitialiser au state initial
store.$reset()
```

8.5 — Setup syntax (Composition API style)

```ts
// src/stores/user.ts — style Composition API
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import type { User } from '@/types'

export const useUserStore = defineStore('user', () => {
  // state
  const currentUser = ref<User | null>(null)
  const isLoading   = ref(false)

  // getters
  const isLoggedIn = computed(() => currentUser.value !== null)
  const displayName = computed(
    () => currentUser.value?.nom ?? 'Invité'
  )

  // actions
  async function login(email: string, password: string) {
    isLoading.value = true
    currentUser.value = await authService.login(email, password)
    isLoading.value = false
  }

  function logout() { currentUser.value = null }

  return { currentUser, isLoading, isLoggedIn, displayName, login, logout }
})
```

8.6 — Actions asynchrones

```
export const usePostsStore = defineStore('posts', {
  state: () => ({
    posts:   [] as Post[],
    loading: false,
    error:   null as string | null
  }),

  actions: {
    async fetchPosts(page = 1) {
      this.loading = true
      this.error   = null
      try {
        const res    = await api.get(`/posts?page=${page}`)
        this.posts   = res.data
      } catch (e) {
        this.error = "Impossible de charger les articles"
        console.error(e)
      } finally {
        this.loading = false
      }
    },

    async createPost(data: NewPost) {
      const post = await api.post('/posts', data)
      this.posts.unshift(post.data)  // ajouter en tête de liste
    }
  }
})
```

8.7 — Composition entre stores

```ts
// src/stores/cart.ts
import { defineStore } from 'pinia'
import { useUserStore } from './user'

export const useCartStore = defineStore('cart', {
  state: () => ({ items: [] as CartItem[] }),

  getters: {
    total: (state) => state.items.reduce(
      (sum, i) => sum + i.prix * i.qty, 0
    )
  },

  actions: {
    async checkout() {
      // Accéder à un autre store depuis une action
      const userStore = useUserStore()
      if (!userStore.isLoggedIn) {
        throw new Error("Connexion requise")
      }
      await api.post('/orders', {
        items:  this.items,
        userId: userStore.currentUser!.id
      })
      this.$reset()
    }
  }
})
```

8.8 — Persistance avec pinia-plugin-persistedstate

```
npm install pinia-plugin-persistedstate
```

```
// main.ts
import { createPinia } from 'pinia'
import piniaPluginPersistedstate from 'pinia-plugin-persistedstate'

const pinia = createPinia()
pinia.use(piniaPluginPersistedstate)
app.use(pinia)
```

```
// stores/auth.ts
export const useAuthStore = defineStore('auth', {
  state: () => ({ token: null as string | null, user: null }),
  // Persiste uniquement 'token' dans localStorage
  persist: {
    key:     'auth',
```

8.9 — Pinia DevTools

Fonctionnalités dans Vue DevTools :

Fonction	Description
Inspection du state	Voir toutes les valeurs en temps réel
Modifier le state	Changer une valeur directement
Timeline des actions	Voir chaque action déclenchée
Time travel	Revenir à un état passé
Déclencher une action	Appeler une action sans code

> 💡 Toutes les actions Pinia sont **traçables** dans la timeline — idéal pour le debugging.

8.10 — Récapitulatif Module 8

Structure

- `defineStore(id, options)`
- `state` — données réactives
- `getters` — valeurs calculées
- `actions` — méthodes sync/async

Utilisation

- `const store = useXxxStore()`
- `storeToRefs()` pour destructurer
- `store.$patch()` pour les maj groupées
- `store.$reset()` pour réinitialiser

✏️ Exercice — Module 8

Exercice 8.1 — Premier store

Exercice 8.2 — Store utilisateur

Exercice 8.3 — Devtools Pinia

> 🕐 Durée estimée : 1h00

Module 9

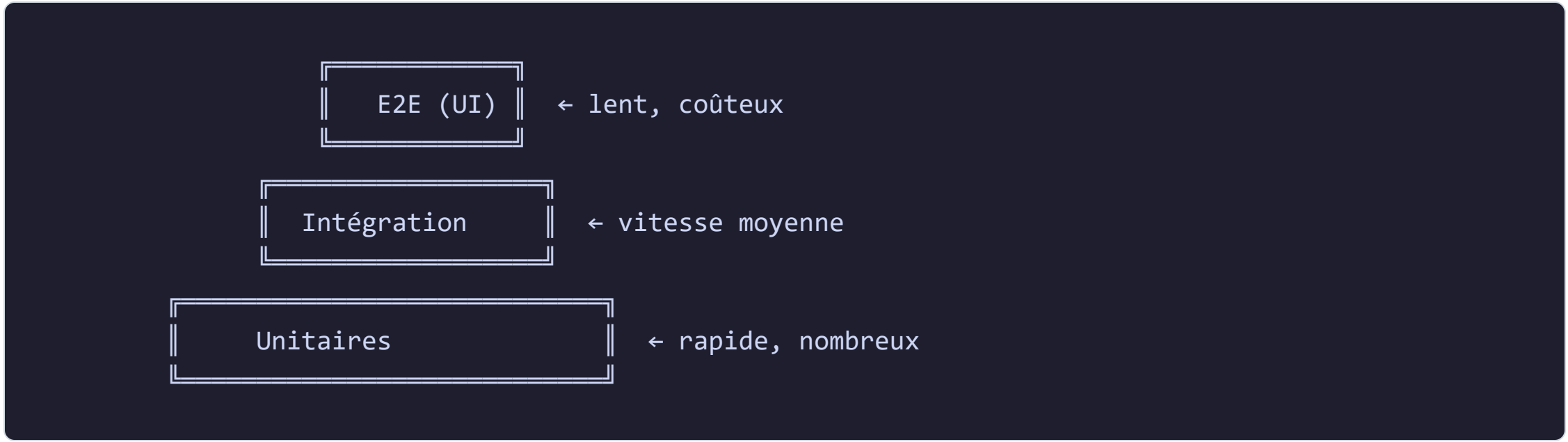
Tests et Qualité du Code

Vitest · Vue Test Utils · Cypress · CI/CD

Module 9 — Plan

1. Pourquoi tester ?
2. Vitest — tests unitaires
3. Vue Test Utils — tests de composants
4. Testing Library
5. Tests E2E avec Cypress
6. Code coverage
7. CI/CD pour Vue.js
8. ✏️ Exercice

9.1 — Pyramide des tests



Type	Quoi tester	Outil
Unitaire	fonctions, composables	Vitest
Composant	composant isolé	Vue Test Utils
E2E	parcours utilisateur complet	Cypress

9.2 — Vitest : configuration et syntaxe

```ts
// vitest.config.ts
import { defineConfig } from 'vitest/config'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  test: {
    environment: 'jsdom',  // simule le navigateur
    globals:     true,     // describe, it, expect sans import
    coverage: {
      reporter:  ['text', 'html'],
      threshold: { lines: 80 }
    }
  }
})
```

```ts
// Structure d'un test
```

9.3 — Vitest : matchers essentiels

```
// Égalité
expect(2 + 2).toBe(4)                    // strict equality
expect({ a: 1 }).toEqual({ a: 1 })       // deep equality
expect(value).not.toBe(null)

// Vérifications
expect(array).toContain("item")
expect(array).toHaveLength(3)
expect(str).toMatch(/pattern/)
expect(fn).toThrow("message d'erreur")

// Truthy / Falsy
expect(true).toBeTruthy()
expect(null).toBeNull()
expect(undefined).toBeUndefined()
expect(0).toBeFalsy()

// Async
await expect(fetchUser(1)).resolves.toEqual({ id: 1 })
await expect(fetchUser(-1)).rejects.toThrow()
```

9.4 — Vue Test Utils : monter un composant

```
import { mount, shallowMount } from '@vue/test-utils'
import UserCard from '@/components/UserCard.vue'

describe('UserCard.vue', () => {
  it('affiche le nom de l\'utilisateur', () => {
    const wrapper = mount(UserCard, {
      props: { nom: 'Alice', age: 28 }
    })
    expect(wrapper.text()).toContain('Alice')
    expect(wrapper.text()).toContain('28')
  })

  it('émet un événement au clic', async () => {
    const wrapper = mount(UserCard, {
      props: { nom: 'Alice', age: 28 }
    })
    await wrapper.find('button').trigger('click')
    expect(wrapper.emitted('select')).toBeTruthy()
    expect(wrapper.emitted('select')![0]).toEqual(['Alice'])
  })
})
```

9.5 — Vue Test Utils : interactions avancées

```
import { mount } from '@vue/test-utils'
import { createTestingPinia } from '@pinia/testing'

it('soumet le formulaire', async () => {
  const wrapper = mount(LoginForm, {
    global: {
      plugins: [createTestingPinia()]  // Pinia de test
    }
  })

  // Remplir un input
  await wrapper.find('[data-testid="email"]')
    .setValue('alice@test.com')
  await wrapper.find('[data-testid="password"]')
    .setValue('motdepasse')

  // Soumettre
  await wrapper.find('form').trigger('submit')

  // Vérifier
  expect(wrapper.emitted('success')).toBeTruthy()
})
```

9.6 — Tester les composables

```ts
// composables/useCounter.test.ts
import { useCounter } from '@/composables/useCounter'

describe('useCounter', () => {
  it('démarre à 0 par défaut', () => {
    const { count } = useCounter()
    expect(count.value).toBe(0)
  })

  it('accepte une valeur initiale', () => {
    const { count } = useCounter(10)
    expect(count.value).toBe(10)
  })

  it('incrémente correctement', () => {
    const { count, increment } = useCounter()
    increment()
    increment()
    expect(count.value).toBe(2)
  })

  it('calcule le double', () => {
    const { count, double, increment } = useCounter()
    increment()
    expect(double.value).toBe(2)
  })
})
```

9.7 — Cypress : tests E2E

```
// cypress/e2e/login.cy.ts
describe('Authentification', () => {
  beforeEach(() => {
    cy.visit('/login')
  })

  it('connecte un utilisateur valide', () => {
    cy.get('[data-cy="email"]').type('alice@test.com')
    cy.get('[data-cy="password"]').type('motdepasse')
    cy.get('[data-cy="submit"]').click()

    cy.url().should('include', '/dashboard')
    cy.contains('Bienvenue Alice').should('be.visible')
  })

  it('affiche une erreur pour identifiants invalides', () => {
    cy.get('[data-cy="email"]').type('inconnu@test.com')
    cy.get('[data-cy="password"]').type('mauvais')
    cy.get('[data-cy="submit"]').click()

    cy.get('.erreur').should('contain', 'Identifiants incorrects')
  })
})
```

9.8 — Code Coverage et seuils qualité

```
# Lancer les tests avec couverture
npm run test:unit -- --coverage

# Résultat :
# ----------------------------|---------|---------|
# File                        | Stmts % | Lines % |
# ----------------------------|---------|---------|
# composables/useCounter.ts   |   100   |   100   |
# components/UserCard.vue     |    85   |    85   |
# stores/auth.ts              |    72   |    72   |
```

```
// vitest.config.ts — seuils minimum
test: {
  coverage: {
    thresholds: {
      lines:      80,
      functions:  80,
```

9.9 — CI/CD avec GitHub Actions

```yaml
# .github/workflows/ci.yml
name: CI Vue.js
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm run test:unit -- --coverage
      - run: npm run build

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: netlify/actions/cli@master
        with: { args: "deploy --prod --dir=dist" }
```

9.10 — Récapitulatif Module 9

Outil	Rôle	Commande
Vitest	Tests unitaires + composables	`npm run test:unit`
Vue Test Utils	Monter et tester des composants	`mount()`
Pinia Testing	Store de test isolé	`createTestingPinia()`
Cypress	Tests E2E navigateur réel	`npx cypress open`
Coverage	Taux de couverture	`vitest --coverage`
GitHub Actions	CI/CD automatisé	`.github/workflows/`

✏️ Exercice — Module 9

Exercice 9.1 — Test unitaire Vitest

Exercice 9.2 — Test de composant

Exercice 9.3 — Test E2E Cypress

Exercice 9.4 — Coverage

> 🕐 Durée estimée : 1h15

🎯 Bilan Jour 2

Ce que vous savez faire

- ✅ Découper une app en composants réutilisables
- ✅ Créer des composables pour la logique partagée
- ✅ Gérer l'état local avec la Composition API
- ✅ Configurer Vue Router avec guards et lazy loading
- ✅ Gérer l'état global avec Pinia
- ✅ Écrire des tests unitaires, composants et E2E
- ✅ Mettre en place un pipeline CI/CD

Demain — Jour 3

📋 Index des slides — Jour 2

Slide	Contenu
1	Couverture
2	Sommaire
3	Module 5 — chapitre
4–14	Architecture composants, composables
15	Récap Module 5
16	Exercice Module 5
17	Module 6 — chapitre
18–32	Composition API, ref, computed, watch
33	Récap Module 6

6.12 — Réactivité avancée : `readonly` et `markRaw`

```
import { readonly, markRaw, reactive } from 'vue'

// readonly — empêche les modifications accidentelles
const config = readonly({ theme: 'dark', lang: 'fr' })
config.theme = 'light'  // ⚠️ Warning en dev, silencieux en prod

// Utile pour les props passées en provide
const state = reactive({ count: 0 })
provide('state', readonly(state))  // consommateurs ne peuvent pas muter

// markRaw — exclure un objet de la réactivité
// Utile pour des objets lourds (cartes, charts, bibliothèques tierces)
import Leaflet from 'leaflet'

const map = markRaw(Leaflet.map('map-container'))
const state2 = reactive({ map })  // map n'est PAS rendu réactif ✅
// → Évite le overhead du tracking sur un gros objet externe
```

6.13 — `effectScope` : grouper les effets

```
import { effectScope, ref, watchEffect } from 'vue'

// Grouper et stopper plusieurs effets d'un coup
const scope = effectScope()

scope.run(() => {
  const count = ref(0)

  watchEffect(() => console.log('A:', count.value))
  watchEffect(() => console.log('B:', count.value * 2))
  // Ces watchers partagent le même cycle de vie
})

// Stopper TOUS les effets du scope en une fois
scope.stop()

// Utile dans les composables complexes
export function useComplexFeature() {
  const scope = effectScope()
  scope.run(() => { /* tous les effets */ })
  onUnmounted(() => scope.stop())
}
```

7.12 — Router : `useLink` et liens actifs

```
import { RouterLink, useLink } from 'vue-router'

// Composant de lien personnalisé avec logique active
const AppLink = defineComponent({
  props: {
    ...RouterLink.props,
    inactiveClass: String,
  },
  setup(props) {
    const { isActive, isExactActive, href, navigate } = useLink(props)

    // Logique personnalisée d'activation
    const isActivated = computed(() => props.to === '/'
      ? isExactActive.value
      : isActive.value
    )

    return { isActivated, href, navigate }
  },
  template: `
    <a :href="href" @click="navigate"
       :class="isActivated ? 'nav-link--active' : 'nav-link'">
      <slot />
    </a>
  `
})
```

7.13 — Scroll behavior et scroll to anchor

```
const router = createRouter({
  history: createWebHistory(),
  routes,

  // Comportement de défilement entre les navigations
  scrollBehavior(to, from, savedPosition) {
    // Restaurer la position sur retour arrière
    if (savedPosition) {
      return savedPosition
    }

    // Scroll vers un ancre (#section)
    if (to.hash) {
      return {
        el:       to.hash,
        behavior: 'smooth',
        top:      80  // offset pour le header fixe
      }
    }

    // Toujours remonter en haut
    return { top: 0, behavior: 'smooth' }
  }
})
```

8.11 — Pinia : abonnements (`$subscribe` et `$onAction`)

```
const store = useCounterStore()

// S'abonner aux changements de state
const unsubscribe = store.$subscribe((mutation, state) => {
  console.log('Type:', mutation.type)   // 'direct' | 'patch object' | 'patch function'
  console.log('Payload:', mutation.payload)
  // Persister à chaque changement
  localStorage.setItem('counter', JSON.stringify(state))
})
onUnmounted(unsubscribe)

// S'abonner aux actions
store.$onAction(({ name, args, after, onError }) => {
  console.log(`Action "${name}" appelée avec`, args)
  after((result) => console.log('Résultat:', result))
  onError((error) => console.error('Erreur:', error))
})
```

8.12 — Pinia : testing avec `createTestingPinia`

```
import { mount }              from '@vue/test-utils'
import { createTestingPinia } from '@pinia/testing'
import { useCounterStore }    from '@/stores/counter'
import MonComposant           from '@/components/MonComposant.vue'

describe('MonComposant avec Pinia', () => {
  it('affiche le count du store', () => {
    const wrapper = mount(MonComposant, {
      global: {
        plugins: [
          createTestingPinia({
            createSpy:    vi.fn,
            initialState: { counter: { count: 42 } }
          })
        ]
      }
    })
    expect(wrapper.text()).toContain('42')
  })

  it('appelle increment au clic', async () => {
    const store = useCounterStore()
    await wrapper.find('button').trigger('click')
    expect(store.increment).toHaveBeenCalled()
  })
})
```

9.11 — Mocking dans les tests

```
import { vi } from 'vitest'

// Mocker un module
vi.mock('@/api/users', () => ({
  usersApi: {
    getAll: vi.fn().mockResolvedValue({
      data: [{ id: 1, nom: 'Alice' }]
    })
  }
}))

// Mocker une fonction
const mockFetch = vi.fn().mockResolvedValue({
  ok:   true,
  json: () => Promise.resolve({ id: 1 })
})
global.fetch = mockFetch

// Espionner sans mocker
const spy = vi.spyOn(console, 'error').mockImplementation(() => {})
// ... test ...
expect(spy).toHaveBeenCalledWith('Message d\'erreur attendu')
spy.mockRestore()
```

9.12 — Tests de snapshot

```
import { mount }  from '@vue/test-utils'
import UserCard   from '@/components/UserCard.vue'

describe('UserCard — snapshots', () => {
  it('correspond au snapshot', () => {
    const wrapper = mount(UserCard, {
      props: { nom: 'Alice', age: 28, role: 'admin' }
    })
    // Crée un snapshot au premier run,
    // compare aux suivants
    expect(wrapper.html()).toMatchSnapshot()
  })

  it('snapshot de la vue admin', () => {
    const wrapper = mount(UserCard, {
      props: { nom: 'Bob', age: 35, role: 'admin', actif: true }
    })
    expect(wrapper.element).toMatchSnapshot()
  })
})

// Mettre à jour les snapshots :
// npx vitest --update-snapshots
```

9.13 — Bonnes pratiques de test

Nommage des tests :

```
// ✅ Clair et comportemental
it('affiche un message d\'erreur si le formulaire est vide')
it('désactive le bouton pendant la soumission')
it('redirige vers /dashboard après connexion réussie')

// ❌ Vague
it('fonctionne')
it('test du bouton')
```

Sélecteurs résistants aux refactoring :

```
<!-- ✅ data-testid — ne change pas si le CSS change -->
<button data-testid="btn-submit">Envoyer</button>

<!-- ❌ Sélecteur CSS fragile -->
```

9.14 — Intégration continue — workflows

```yaml
# Workflow complet avec cache npm
name: Tests & Lint
on: [push, pull_request]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'              # cache des node_modules
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check    # vue-tsc --noEmit
      - run: npm run test:unit -- --coverage --reporter=verbose
      - run: npm run build
      - uses: codecov/codecov-action@v3  # rapport de coverage
```

5.10 — Barrel exports — organisation des imports

```
// components/ui/index.ts — export centralisé
export { default as AppButton }   from './AppButton.vue'
export { default as AppInput }    from './AppInput.vue'
export { default as AppModal }    from './AppModal.vue'
export { default as AppBadge }    from './AppBadge.vue'
export { default as AppSpinner }  from './AppSpinner.vue'

// components/index.ts
export * from './ui'
export * from './forms'
export * from './layouts'

// Dans n'importe quel fichier — import propre et simple
import { AppButton, AppModal, AppInput } from '@/components'
// Au lieu de :
import AppButton from '@/components/ui/AppButton.vue'
import AppModal  from '@/components/ui/AppModal.vue'
```

6.14 — Patterns réactifs avancés

```
// Debounced ref — version simplifiée
function useDebouncedRef<T>(value: T, delay = 200) {
  let timer: ReturnType<typeof setTimeout>
  return customRef((track, trigger) => ({
    get() { track(); return value },
    set(newValue: T) {
      clearTimeout(timer)
      timer = setTimeout(() => {
        value = newValue
        trigger()
      }, delay)
    }
  }))
}

const rechercheDebounced = useDebouncedRef('', 500)
// Utilisation identique à une ref normale
// mais les watchers ne se déclenchent qu'après 500ms
```

7.14 — Route resolving et redirections

```
const routes = [
  // Redirection simple
  { path: '/home', redirect: '/' },

  // Redirection nommée
  { path: '/profile', redirect: { name: 'user-profile' } },

  // Redirection dynamique avec fonction
  {
    path: '/old/:id',
    redirect: (to) => ({
      name: 'new-route',
      params: { newId: to.params.id }
    })
  },

  // Alias — plusieurs URL pour la même vue
  {
    path: '/users',
    alias: ['/membres', '/equipe'],
    component: UsersView
  }
]
```

8.13 — Optimistic updates avec Pinia

```
export const usePostsStore = defineStore('posts', {
  state: () => ({ posts: [] as Post[] }),

  actions: {
    async likerPost(postId: number) {
      // 1. Mise à jour optimiste — UI réagit immédiatement
      const post = this.posts.find(p => p.id === postId)
      if (!post) return
      post.likes++
      post.likedByMe = true

      try {
        // 2. Envoyer au serveur
        await api.post(`/posts/${postId}/like`)
      } catch (error) {
        // 3. Rollback si erreur
        post.likes--
        post.likedByMe = false
        this.error = "Impossible d'ajouter le like"
      }
    }
  }
})
```

9.15 — Testing accessibility (a11y)

```
import { mount }          from '@vue/test-utils'
import { axe, toHaveNoViolations } from 'jest-axe'

expect.extend(toHaveNoViolations)

describe('Accessibilité', () => {
  it('Formulaire de login sans violations a11y', async () => {
    const wrapper = mount(LoginForm)
    const results = await axe(wrapper.element)
    expect(results).toHaveNoViolations()
  })

  it('La modal est accessible au clavier', async () => {
    const wrapper = mount(AppModal, {
      props: { visible: true, titre: 'Confirmation' }
    })
    // Vérifier les attributs ARIA
    expect(wrapper.find('[role="dialog"]').exists()).toBe(true)
    expect(wrapper.find('[aria-modal="true"]').exists()).toBe(true)
    expect(wrapper.find('[aria-labelledby]').exists()).toBe(true)
  })
})
```